

Workflows bauen, die laufen

Ein Konzeptvorschlag fuer das Erstellen und Optimieren mit dem Copilot

Entwurf auf Basis der Ursachenanalyse vom 30. Juli 2026 · Diskussionsgrundlage, keine Implementierung

Dieses Papier schlaegt vor, wie der Copilot vom heutigen „Bauen und Reparieren“ zu einem „Erkunden, Bauen, Lernen“ kommt. Es baut auf der Analyse der 31 Produktionssitzungen auf und nutzt bewusst Bausteine, die bereits im System stecken — sie werden heute nur nicht verwendet.

Die Ausgangslage in drei Saetzen

Von 31 autonomen Optimierungslaeufen war keiner erfolgreich; jeder wurde von Hand abgebrochen. Die Laeufer scheiterten nicht am Budget, sondern an Selektoren — also an der Frage, welches Element auf der Seite gemeint ist. Genau das ist die Disziplin, in der der Copilot heute am schwaechsten ist, weil er Struktur repariert statt Selektoren.

Die Kernthese

Der Copilot plant heute blind: Er entwirft einen Workflow, bevor er die Seite gesehen hat, und korrigiert danach die Struktur — Routen, Reihenfolgen, Abzweigungen. Der tatsaechliche Fehler liegt aber fast immer eine Ebene tiefer, im Selektor. Deshalb repariert er zuverlaessig die falsche Sache. Und weil nichts von dem, was er unterwegs herausfindet, gespeichert wird, beginnt jeder Lauf wieder bei null.

Was daraus folgt

Nicht die Reparaturlogik braucht mehr Intelligenz, sondern der Weg davor: erst die Seite kennen, dann bauen — und behalten, was funktioniert hat.

Der Vorschlag: sechs Bausteine

Die Bausteine greifen ineinander, lassen sich aber einzeln einfuehren. A und B liefern den groessten Teil der Wirkung.

A Selektoren pruefen, bevor der Workflow laeuft

Die Zielseite einmal oeffnen und jeden Selektor des Workflows dagegen halten. Drei Ergebnisse zaehlen: kein Treffer (der Selektor ist falsch), genau ein Treffer (gut), mehrere Treffer (zu unspezifisch — genau der Fall `button:has-text("Decline")`, der produktiv blockierte). Der Lauf startet erst, wenn jeder Selektor genau einmal trifft. Heute pruefte der Preflight nur die Definition auf Vollstaendigkeit, nicht die Selektoren gegen die echte Seite.

B Ein Gedaechnis fuer bewaehrte Selektoren

Was einmal funktioniert hat, wird pro Domain und Zweck gespeichert: `google.com + Suchfeld = textarea[name="q"]`. Neue Workflows schlagen dort zuerst nach, statt zu raten. Die noetigen Daten entstehen bereits heute in jeder Beobachtung (`selector_candidates` in der Interaktionskarte) — sie werden nur nach dem Lauf verworfen. Das ist der fehlende Lerneffekt: Ohne ihn beginnt jede Sitzung wieder bei null.

C Erst erkunden, dann planen

Vor dem Bauen eine kurze Erkundung: Seite oeffnen, Elemente kartieren, Consent-Situation erkennen, Formularfelder benennen. Aus dieser Karte entsteht der Workflow — mit echten Selektoren statt geratener. Der Copilot kennt diesen Schritt schon als Probe, setzt ihn aber nur reaktiv zur Reparatur ein, nicht vorab zum Bauen.

D Fortschritt messen statt nur urteilen

Das Vision-Modell liefert bereits einen Zielfortschritt von 0 bis 1 — dieser Wert wird heute protokolliert und dann weggeworfen. Als Regelgroesse verwendet, beantwortet er die wichtigste Frage einer Reparatur: Ist der Lauf danach naeher am Ziel als davor? Wird es schlechter, wird die Aenderung zurueckgenommen statt darauf aufgebaut.

E Das Ziel maschinenlesbar erfassen

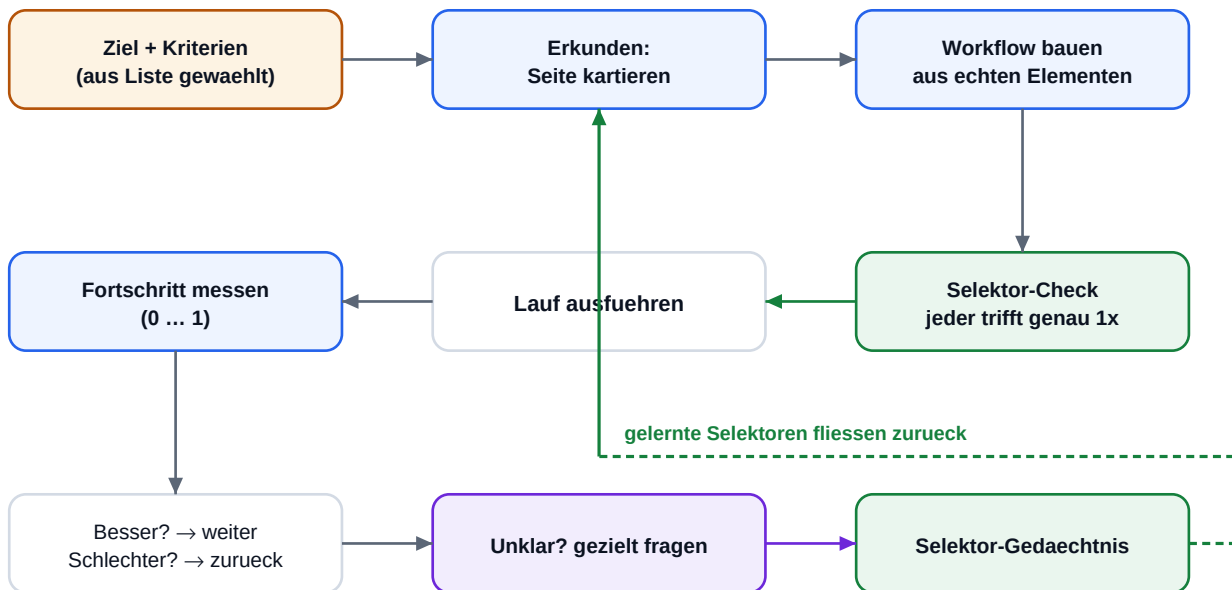
Erfolgskriterien werden heute frei eingetippt. Produktiv standen dort Werte wie „sassion speichern“ oder „workflow_return = array“ — beide fallen still auf „nicht pruefbar“, und ohne gueltiges Kriterium gilt die fachliche Pruefung als bestanden, ohne je etwas geprueft zu haben. Besser: Kriterium aus einer Liste waehlen (URL enthaelt, Text sichtbar, Rueckgabewert ist Array) und den Wert dazu eintragen. Freitext bleibt moeglich, wird aber sofort als ungueltig markiert.

F Gezielt fragen statt still zu pausieren

Wenn der Copilot nicht weiterkommt, pausiert er heute und wartet. Besser waere eine konkrete Frage mit Screenshot: „Ich finde den Ablehnen-Knopf nicht — ist es dieser hier?“ Ein Klick genuegt, der Lauf geht weiter, und die Antwort wandert in das Gedaechnis aus Baustein B. Aus einer Sackgasse wird so ein Lernmoment.

Wie der Ablauf dann aussieht

Der grosse Unterschied: Die Schleife dreht sich nicht mehr um „Struktur reparieren“, sondern um „Selektor bestaetigen“ — und jede Erkenntnis wird behalten.



Einfuehrung in drei Schritten

- Schritt 1 — Baustein A und E. Der Selektor-Check verhindert den haeufigsten Fehlertyp, die gefuehrte Zieleingabe den unsichtbarsten. Beide sind fuer sich nutzbar und aendern nichts an der bestehenden Laufsteuerung.
- Schritt 2 — Baustein B und F. Das Gedaechnis braucht eine kleine Tabelle (Domain, Zweck, Selektor, Trefferquote); die gezielte Rueckfrage fuehrt sie mit menschlichem Wissen. Ab hier wird jeder Lauf besser als der vorige.
- Schritt 3 — Baustein C und D. Erkundung vor Planung und Fortschritt als Regelgrosse sind die groesseren Umbauten — sie lohnen sich, sobald die ersten beiden Schritte greifen.

Abwaegungen und Grenzen

Jeder Vorschlag kostet etwas. Diese Punkte sollten vor einer Entscheidung auf dem Tisch liegen.

Was der Vorschlag kostet

- Der Selektor-Check (A) macht den Start langsamer — eine Seite muss vorab geladen werden. Dafuer entfaellt der haeufigste Grund fuer stundenlange Fehlversuche.
- Das Gedaechnis (B) ist neuer, dauerhafter Zustand: eine Tabelle, die gepflegt und bei Seitenaenderungen entwertet werden muss. Ein Selektor, der gestern stimmte, kann heute falsch sein — deshalb sollte jeder Eintrag eine Trefferquote und ein Datum tragen und bei wiederholtem Fehlschlag automatisch verfallen.
- Die gefuehrte Zieleingabe (E) schraenkt ein. Wer heute frei formuliert, muss sich kuenftig auf pruefbare Muster festlegen. Das ist gewollt, wird aber als Einschraenkung wahrgenommen — der Freitext sollte deshalb erhalten bleiben und nur klar als „nicht pruefbar“ markiert werden.
- Die Rueckfrage (F) bricht die Autonomie bewusst auf. Das ist ein Feature, kein Mangel: Ein gezielt gefragter Klick ist billiger als vierzig Minuten Reparaturschleife.

Was der Vorschlag nicht loest

Seiten mit aktivem Bot-Schutz, Zwei-Faktor-Anmeldungen oder haeufig wechselndem Aufbau bleiben schwierig. Auch das beste Selektor-Gedaechnis hilft nicht, wenn sich das Ziel bei jedem Aufruf aendert. Fuer solche Faelle ist die ehrliche Antwort ein frueher, klarer Abbruch mit Begruendung — nicht ein weiterer Reparaturversuch.

Bereits umgesetzt (30. Juli 2026)

Unabhaengig von diesem Konzept wurden drei Selbstblockaden behoben: Elementdaten werden beim Pausieren nicht mehr pauschal verworfen, wiederholte Reparaturplaene werden zuverlaessig erkannt, und es gibt eine harte Obergrenze fuer Checkpoint-Reparaturen. Damit endet ein aussichtsloser Lauf nach Minuten statt nach Stunden. Das Konzept oben setzt darauf auf.

Vorschlag zum Weitermachen

Baustein A an einem einzigen realen Workflow ausprobieren — idealerweise dem Google-Suche-Workflow, dessen Selektoren aus den Protokollen bereits bekannt sind. Wenn der Check dort die bekannten Fehltreffer findet, ist der Nutzen belegt, bevor groesser gebaut wird.